



Python程序设计

一、基础知识

学习内容：

1. **数字类型**：运算符 函数 属性与方法 内置函数
2. **函数定义与调用**

大纲

- Python语言概述和安装
- Python基础知识：基本的输入输出
- Python基础知识：变量和对象
- Python基础知识：数字类型
- Python基础知识：模块和包
- Python代码编写规范
- Python基础知识：字符串与更多的输出选项
- Python编程快速入门

1.4.3 数字

- 数字为不可变对象
- 包括整数(int)、浮点数(float)、复数(complex)、布尔 (bool)
- 与有些语言不同，整数可以表示任意大的数值，具体可以大到什么程度仅受内存大小的限制。

```
>>> 9999 ** 99                                     #这里**是幂乘运算符，等价于内置函数pow()  
990148353526723487602263124753282625570559528895791057324326529121794837894053  
513464422176826916433932586924386677766244032001623756821400432975051208820204  
980098735552703841362304669970510691243800218202840374329378800694920309791954  
185117798434329591212159106298699938669908067573374724331208942425544893910910  
073205049031656789220889560732962926226305865706593594917896276756396848514900  
989999
```

整数

- 整数**字面值** (integer literal)
 - 十进制整数: 0、-1、9、123 注意**007不可以**
 - 十六进制整数: **0x**开头后面为十六进制数字, 即为0-9以及[a-f] (a、b、c、d、e、f) 如 0x10、0xfa、0xabcdef,注意这些数字的**大写字母(包括前缀0X)**也可以
 - 八进制整数:以**0o**开始后面为八进制数字即0-8, 比如0o35、0o11
 - 二进制整数:以**0b**开头如, 0b101、0b100 、 0b10010111
- Python 3.6.x支持在数字中间位置使用**单个下划线**作为分隔来提高数字的可读性, 类似于数学上使用逗号作为千位分隔符。数字中单个下划线可以出现在**中间任意位置**, 但不能出现开头和结尾位置, 也**不能使用多个连续的下划线**。

```
>>> 1_000_000  
1000000
```

```
>>> 1_2_3_4  
1234
```

```
>>> 1_2.3_45 #实数也可以这样写  
12.345
```

```
>>> 1_2 + 3_4j #复数也可以这样写  
(12+34j)
```

(有限精度) 浮点数 (float)

- **浮点数字面值**：浮点数用于表示实数，在内部采用科学计数法表示，因此称为浮点数
 - 小数表示：3.14 10. .001
 - 科学计数法：15e-2 ($=15 \times 10^{-2} = 0.15$) 3.14e-10 1.0E100
 - 需要说明的是由于计算机内部采用二进制表示，与其他语言一样，采用IEEE 574双精度表示浮点数时，有精度误差

```
>>> 0.4 - 0.1
```

#实数相减，结果稍微有点偏差

```
0.30000000000000004
```

```
>>> 0.4 - 0.1 == 0.3
```

#**应尽量避免直接比较两个实数是否相等**

```
False
```

```
>>> abs(0.4-0.1 - 0.3) < 1e-6
```

#这里1e-6表示10的-6次方

```
True
```

- Python提供了**decimal**模块（高精度浮点数Decimal）用于十进制数学计算，来保证用户指定的精度

```
>>> 1 / 9
```

#内置的实数类型

```
0.11111111111111111
```

```
>>> from decimal import *
```

```
>>> Decimal(1/9)
```

```
Decimal('0.1111111111111111104943205418749130330979824066162109375')
```

```
\>>>
```

分数

- Python标准库fractions中的Fraction对象支持分数及其运算。

```
>>> from fractions import Fraction
```

```
>>> x = Fraction(3, 5)      #创建分数对象
```

```
>>> y = Fraction(3, 7)
```

```
>>> x
```

```
Fraction(3, 5)
```

```
>>> x ** 2                  #幂运算
```

```
Fraction(9, 25)
```

```
>>> x.numerator            #查看分子, 对象的属性, 格式: 对象.属性
```

```
3
```

```
>>> x.denominator          #查看分母
```

```
5
```

```
>>> x + y                  #支持分数之间的四则运算, 自动进行通分
```

```
Fraction(36, 35)
```

复数 (Complex)

■ Python内置支持复数类型及其运算，并且形式与数学上的复数完全一致。

```
>>> x = 3 + 4j
```

#使用j或J表示复数虚部 注意 j 4*j 都不是复数

```
>>> y = 6j + 5
```

```
>>> x + y
```

#支持复数之间的加、减、乘、除以及幂乘等运算

```
(8+10j)
```

```
>>> x * y
```

```
(-9+38j)
```

```
>>> abs(x)
```

#内置函数abs()可用来计算复数的模

```
5.0
```

```
>>> x.imag
```

#虚部 对象的属性，格式：对象.属性

```
4.0
```

```
>>> x.real
```

#实部 对象的属性，格式：对象.属性

```
3.0
```

```
>>> x.conjugate()
```

#共轭复数 对象的方法，格式：对象.方法

```
(3-4j)
```

布尔 (bool)

- 整数类型的特例，取值为True、False，分别对应整数1和0
- 实际上所有非0的整数转变为bool时都是True
- True 等价于 非0，False等价于0
- `int(True)=1`, `int(False) = 0`
- `bool(4)=True`, `bool(0) = False`

```
>>> True + 1
2
>>> False * 4
0
```

1.4.5 运算符和表达式

算术运算符

比较运算符

逻辑运算符

序列

运算符示例	功能说明
x+y	算术加法，列表、元组、字符串合并
x-y	算术减法，集合差集
x*y	乘法，序列重复
x/y	除法（在Python 3.x中叫做真除法）
x//y	求整商
-x	相反数
x%y	余数（对实数也可以进行余数运算），字符串格式化
x**y	幂运算
x<y x<=y x>y x>=y	大小比较（可以连用），集合的包含关系比较
x==y x!=y	相等（值）比较，不等（值）比较
x or y	逻辑或（只有x为假才会计算y）
x and y	逻辑与（只有x为真才会计算y）
not x	逻辑非
x in y x not in y	成员测试运算符
x is y x is not y	对象实体同一性测试（地址）（已经学了）
^ & << >> ~	位运算符（不作要求）
& ^	集合交集、并集、对称差集

算术运算符

- + - 加法、减法： $35+4$ $2.5 + 3.5$
- 乘法*： $(35+4)*5$, $3.4*x + 4$ 注意*不能省略
- 幂运算 ** $4**2 = 16$ $4**0.5 = 2.0$
- 一元的改变符号 +- $+8$ -8
- 除法：
 - 浮点除法/（也称“真除法”），结果为浮点数 $5/2 = 2.5$ $10/3.2 = 3.125$
 - 整除// $5//2 = 2$ $-5//2 = -3$ $5.1//2.2 = 2.0$ 为向负无穷方向舍入（floor()函数）
 - $2/0$ $2//0$ run-time error
- 取余（求模） %
 - $a \% b = a - c * b = a - a // b * b$ 这样在其结果的符号与b一致
 - $10\%5 = 0$ $20\%3 = 2$ $-20\%3 = 1$

求模 VS. 求余运算: %

- **求模(modulo)**: $a \% b = a - c * b = a - a // b * b$
- **求余(remainder)**与求模运算类似, 特别是 a 和 b 符号一致时两者结果一样, 但是 a 和 b 符号不一致时结果不一样
 - Python支持求模运算, 其结果的符号与 b 一致
 - C/C++、Java为求余运算, 其结果的符号与 a 一致
- 存在这个区别的原因是在于在计算整商时, **求模运算为向负无穷方向舍入(floor()函数)**, 而求余运算为向原点(0)方向舍入(trunc()函数)

```
>>> print(5 // 2, -5 // 2, 5 // -2, -5 // -2)
2 -3 -3 2
>>> print(5 % 2, -5 % 2, 5 % -2, -5 % -2)
1 1 -1 -1
```

$5 // 2 = 2$	$5 \% 2 = 1$
$-5 // 2 = -3$	$-5 \% 2 = 1$
$5 // -2 = -3$	$5 \% -2 = -1$
$-5 // -2 = 2$	$-5 \% -2 = -1$

算术运算符一些特性

- 优先级顺序，下面四组从低到高，同级的左结合运算

- +- 最低

- * / // %

$$3-4**2+10\%5+8*6//7 = 3 - 16 + 0 + 48 // 7 = -7$$

- 一元的+-

- **

- 通过小括号改变运算顺序，可嵌套，注意：中括号[]和大括号{}不用于数学计算
- 上述运算符支持两个不同的数字类型的对象之间的运算，即混合运算，比如浮点数+整数时会进行隐式的类型转换
 - 如果其中包含complex对象，则其他数字类型转换为complex对象，结果为complex对象
 - 如果其中包含float对象，则其他数字类型转换为float对象，结果为float对象

bool int → float → complex

复旦大学版权所有

比较运算符

- 比较数字对象的值的大小，结果为bool值True或者False
 - $x < y$; $x \leq y$; $x > y$; $x \geq y$
 - 可以连用，连用时**不考虑传递性**，仅仅相邻数之间的比较
 - $x < y < z$ 表示 $x < y$ **且** $y < z$
 - $x < y > z$ 表示 $x < y$ **且** $y > z$
- ```
>>> x,y,z=12,18,13
>>> x < y, x <= y, x > y, x >= y # (True, True, False, False)
>>> x < y < z, x < y > z # (False, True)
```
- 比较对象的值是否相等
  - $x == y$ 

```
>>> x == y
```

 # False
  - $x \text{ **!=** } y$ 

```
>>> x != y
```

    # True

## 例子：求模的应用（学过循环后要求掌握）

- 判别奇偶数： $n \% 2 == 0$
- 判断n是否能够被m整除： $n \% m == 0$
- 今天星期几(start in [0,6]), 在n天之后是星期几？ $(start + n) \% 7$
- 每行输出m个数后换一行：
- 获得整数n的各位数字

```
count = 0
m = 10
for i in range(100):
 print(i, end='\t')
 count += 1
 if count % m == 0:
 print()
```

每行输出m个数后换一行

```
def digitalNum(num):
 place = 1 # 右边数起第几位
 while True:
 digit = num % 10
 print('右边数起第%d位数字为%d' %
 (place, digit))
 place = place + 1
 num = num // 10
 if num == 0:
 return
```

num=134

digitalNum(num)

获得整数n的各位数字

# 内置数学函数

```
m = max(4,5,6)
n = min(4,5,6)
print(m,n)
```

| 函数                      | 含义                           | 实例                                 | 结果                      |
|-------------------------|------------------------------|------------------------------------|-------------------------|
| abs(x)                  | x的绝对值。如果x为复数，则返回其模           | abs(-1.2)<br>abs(1-2j)             | 1.2<br>2.23606797749979 |
| divmod(a,b)             | 返回a除以b的商和余数                  | divmod(5,3)                        | (1,2)                   |
| pow(x,y[,z])            | 返回x**y。如果z有，则为pow(x,y)%z     | pow(2,10)<br>pow(2,10,10)          | 1024<br>4               |
| round(number[,ndigits]) | 四舍五入取整，如果ndigits则保留ndigits小数 | round(3.14159)<br>round(3.14159,4) | 3<br>3.1416             |
| max(arg1,arg2,*args)    | 取最大值                         | max(1,7,3,15,14)                   | 15                      |
| min(arg1,arg2,*args)    | 取最小值                         | min(1,7,3,15,14)                   | 1                       |
| sum(iterable[,start])   | 求和，start如果有，表示加上start        | sum((1,2,3))<br>sum((1,2,3),44)    | 6<br>50                 |

# 第5章 函数

- 在实际开发中，有许多操作时完全相同或非常相似的，仅仅是要处理的数据不同，为此需要实现**代码复用**。
- 将可能需要反复执行的代码封装为函数，并在需要该功能的地方进行调用，不仅可以实现**代码复用**，更重要的是可以**保证代码的一致性**，只需要修改该函数代码则所有调用均受到影响。
- Python提供了许多常用的内置函数我们已学习print()、input()等
- **如何定义自己的函数呢？**

# 函数定义

**def** 函数名([形参列表]):

'''注释'''

函数体

- ✓ 函数体的开始以**def**开始，然后是一个空格和函数的名称，接下来是一对圆括号
- ✓ 圆括号内是形式参数列表，称为**形参**，**形参**如果有多个以逗号(,)隔开，参数可以没有，也**必须**保留一对空的圆括号
- ✓ 括号后面的**冒号**必不可少
- ✓ 与其它语言不同，函数形参**不需要**声明类型，也**不需要**指定函数返回值类型
- ✓ Python**允许嵌套定义函数**

```
def fahrenheit_from_Celsius(Celsius):
```

```
 """摄氏度转换到华氏度"""
```

```
 Fahrenheit = (9 / 5) * Celsius + 32
```

```
 return Fahrenheit
```

```
temperature_cel = 35
```

```
temperature_fah = fahrenheit_from_Celsius(temperature_cel)
```

```
print(temperature_fah)
```

```
def my_sum(start, stop, step):
```

```
 """ 求和, 从start到stop (不包括) , 步长为step """
```

```
 i = start
```

```
 sum = 0
```

```
 while i < stop:
```

```
 sum += i
```

```
 i += step
```

```
 return sum
```

```
sumValue = my_sum(1, 10, 3)
```

```
print(sumValue)
```

# 函数定义：文档字符串

def 函数名([形参列表]):

'''注释'''

函数体

- 定义函数（及模块）时，开头部分的**注释**并不是必需的，但是如果为函数的定义加上这段注释的话，可以为用户提供友好的提示和使用帮助。
- 该字符串保存在函数（及模块）对象的\_\_doc\_\_属性中，称为docstring
- help(func)可以查看相应的\_\_doc\_\_属性
- IDLE界面会在相应的calltip给出其注释

```
def fahrenheit_from_celsius(celsius):
 """ 摄氏度转换到华氏度 """
 fahrenheit = (9 / 5) * celsius + 32
 return fahrenheit
```

```
>>> help(fahrenheit_from_celsius)
Help on function fahrenheit_from_celsius in module __main__:

fahrenheit_from_celsius(celsius)
 摄氏度转换到华氏度

>>> print(fahrenheit_from_celsius(
 (celsius)
 摄氏度转换到华氏度
```

# 函数定义：函数体

def 函数名([形参列表]):

"""注释"""

函数体

- 函数体一般包括多行，相对于def关键字必须保持一定的空格缩进，是函数执行的代码块
- 如果函数体比较简单时，也可以与def在同一行
- 函数体可以只写一句pass语句为空语句，经常用在定义一个空函数，以后再扩充
- 函数可以返回值，也可以不返回。
  - 通过return语句返回对象
  - 如果函数体中包含return语句，不论return语句出现在函数的什么位置，一旦得到执行将直接结束函数的执行（即后面的代码不再执行），并同时返回return语句中相应表达式所创建的对象；如果没有return，返回的值为None，其类型为NoneType（只有一个值即None）
  - 如果执行到函数结束也无return语句，则等价于 return None

```
def fahrenheit_from_celsius(celsius):
 """ 摄氏度转换到华氏度 """
 fahrenheit = (9 / 5) * celsius + 32
 return fahrenheit

def cubic(n): return n**3

def printName(name):
 """将姓名打印出来"""
 print("我的名字是:", name)
```

# 函数：函数调用

- 函数调用：

函数名([实参列表])

fahrenheit\_from\_Celsius(temperature\_cel)

```
def fahrenheit_from_Celsius(Celsius):
 """摄氏度转换到华氏度"""
 Fahrenheit = (9 / 5) * Celsius + 32
 return Fahrenheit
```

```
temperature_cel = 35
temperature_fah = fahrenheit_from_Celsius(temperature_cel)
print(temperature_fah)
```

- 实参(argument)：应该与形参（函数定义时的参数）对应，如有多个参数之间以逗号分隔，表示传递实参给出的对象引用到相应的形参，即函数调用时向其传递实参，python采用赋值传递(pass by assignment)的策略

**形参变量=实参变量**，即形参变量与实参变量指向同一个对象

- 绝大多数情况下，在函数内部直接修改形参的值不会影响实参
  - 对于不可变对象，显然如此
  - 对于可变对象，需要小心处理
- 为了避免side effect，函数中的形参（变量）的使用范围为函数体(作用域部分会详细介绍)

# 例：赋值传递

```
def myFun(aNum,aStr,aList):
 """为了演示形参和实参之间的赋值传递"""
 print(aNum,aStr,aList)
 #证明形参和实参是指向相同的对象
 print("aNum's ID :",id(aNum))
 print("aStr's ID :",id(aStr))
 print("aList's ID :",id(aList))

 #对于可变对象，因为指向同一个对象，所以可以在函数中修改该实参对象
 aList.append("a new element")#为列表增加一个新的元素

 #将返回下面创建的新的列表
 aResult =[aNum]+[aStr]+[aList]
 print("aResult's ID :",id(aResult))
 print("I will return this list:",aResult)
 return aResult
```



```
>>> num1 = 2
>>> str1 ="hello"
>>> list1= [1,2,3]
>>> id(num1)
4376852448
>>> id(str1)
4551955936
>>> id(list1)
4551872776
>>> returnList = myFun(num1,str1,list1) #aNum = num1 aStr=str1 alist =list1
2 hello [1, 2, 3]
aNum's ID : 4376852448
aStr's ID : 4551955936
aList's ID : 4551872776
aResult's ID : 4551949320
I will return this list: [2, 'hello', [1, 2, 3, 'a new element']]
>>> returnList
[2, 'hello', [1, 2, 3, 'a new element']]
>>> id(returnList)
4551949320
>>>
```

# 位置参数

- 位置参数 (positional arguments) 是比较常用的形式，函数定义时只包含形参名的参数称为**位置 (positional) 参数**，调用函数时**实参和形参的顺序必须严格一致**，并且**实参和形参的数量必须相同**，形参根据其出现的位置顺序来逐个匹配相应的实参

```
def my_sum(start, stop, step):
 """ 求和, 从start到stop (不包括), 步长为step """

 i = start
 sum = 0
 while i < stop:
 sum += i
 i += step
 return sum

sumValue = my_sum(1, 10, 3)
print(sumValue)
```

```
>>> def demo(a, b, c):
 print(a, b, c)
>>> demo(3, 4, 5) #按位置传递参数
3 4 5
>>> demo(3, 5, 4)
3 5 4
>>> demo(1, 2, 3, 4) #实参与形参数量必
须相同
TypeError: demo() takes 3 positional arguments but 4
were given
```

# 位置参数

- 位置参数 (positional arguments) 是比较常用的形式，函数定义时只包含形参名的参数称为**位置 (positional) 参数**，调用函数时**实参和形参的顺序必须严格一致**，并且**实参和形参的数量必须相同**，形参根据其出现的位置顺序来逐个匹配相应的实参

```
def my_sum(start, stop, step):
 """ 求和, 从start到stop (不包括), 步长为step """

 i = start
 sum = 0
 while i < stop:
 sum += i
 i += step
 return sum

sumValue = my_sum(1, 10, 3)
print(sumValue)
```

```
>>> def demo(a, b, c):
 print(a, b, c)
>>> demo(3, 4, 5) #按位置传递参数
3 4 5
>>> demo(3, 5, 4)
3 5 4
>>> demo(1, 2, 3, 4) #实参与形参数量必
须相同
TypeError: demo() takes 3 positional arguments but 4
were given
```

# 默认值参数

- Python在定义函数时，支持**默认值参数**，即可为形参设置默认值(缺省值)。

```
def 函数名(……, 形参名=默认值):
 函数体
```

- 调用带有**默认值参数**的函数时，可以不对默认值参数进行赋值，也可以赋值，具有较大的灵活性。
- 使用“**函数名.\_\_defaults\_\_**”以元组的形式查看函数所有默认值参数的当前值。

```
>>> def say(message, times =1):
 print (message * times)
```

```
>>> say('hello')#不为默认值参数传值
hello
```

```
>>> say('hello', 3)#使用调用者显示传递的值
hellohellohello
```

```
>>> say('hi', 7)#使用调用者显示传递的值
hihihihihihihihihihi
```

```
>>> say.__defaults__
(1,)
```

# 默认值参数(defaults parameter)

- 函数定义时最前面是位置参数，然后是默认值参数
- 即默认值参数右边不能再有位置参数。
- 为什么这样设计？调用时：有默认值的参数可以省略；这样才好匹配：首先普通的位置参数必须的进行匹配，剩下的如果实参有，则采用实参的值；如果没有，则采取默认值

```
>>> def f(a=3,b,c=5):
 print a,b,c
```

```
SyntaxError: non-default argument follows default argument
```

```
>>> def f(a=3,b):
 print a,b
```

```
SyntaxError: non-default argument follows default argument
```

```
>>> def f(a,b,c=5):
 print a,b,c
```

```
>>>
```

# 默认值参数(defaults parameter):示例

```
def my_sum(start, stop, step):
 """ 求和, 从start到stop (不包括), 步长为step """
```

```
def my_sum_default(start, stop, step=1, init=0):
 """ 求和, 从start到stop (不包括), 步长为step, 缺省为1, 和最后再加上init, 缺省为0 """
```

```
 i = start
 sum = init
 while i < stop:
 sum += i
 i += step
 return sum
```

```
print(my_sum_default(0, 101, 1, 0))
print(my_sum_default(0, 101, 1))
print(my_sum_default(0, 101))
print(my_sum_default(0, 101, 1, 1000))
print(my_sum_default(0, 101, 2))
```

如果step取默认值, 而想改变init默认值, 怎么办?

# 关键字参数(keyword argument)

- 关键字参数主要指**实参**，即**调用函数时**的参数传递方式。
- 通过**关键字参数可以按参数名字**传递值，实参顺序可以和形参顺序不一致，但不影响传递结果，避免了用户需要牢记位置参数顺序的麻烦。
- 注意**关键字参数**与函数定义时的**缺省值参数**的区别，虽然两者形式一致，但两者之间**并不等同**：**一个是在函数调用时，一个是在函数定义的时候**
- 调用时可以使用位置参数和关键字来传递参数，但是与函数定义时类似，**关键字参数右边不能有位置参数**

```
>>> def demo(a, b, c=5):
 print (a, b, c)
>>> demo(3, 7)
3 7 5
>>> demo(a=7, b=3, c=6)
7 3 6
>>> demo(c=8, a=9, b=0)
9 0 8
```

# 关键字参数(keyword argument)

- 在函数定义中有多个缺省值参数时

通过关键字参数来明确传递哪些参数，哪些采用默认值

```
def func(a, b, c=5, d=6): print(a, b, c, d)
func(1, 2, 7)
func(1, 2, 5, 8)
func(b=1, a=2, c=7)
func(1, 2, d=8)
func(1, b=2, d=8)
```

```
def my_sum_default(start, stop, step=1, init=0):
 """ 求和, 从start到stop (不包括), 步长为step,
 缺省为1, 和最后再加上init, 缺省为0"""
 i = start
 sum = init
 while i < stop:
 sum += i
 i += step
 return sum

print(my_sum_default(0, 10, init=100))
```

# 内置数学函数

```
m = max(4,5,6)
n = min(4,5,6)
print(m,n)
```

| 函数                      | 含义                                                                    | 实例                                 | 结果                      |
|-------------------------|-----------------------------------------------------------------------|------------------------------------|-------------------------|
| abs(x)                  | x的绝对值。如果x为复数，则返回其模                                                    | abs(-1.2)<br>abs(1-2j)             | 1.2<br>2.23606797749979 |
| divmod(a,b)             | 返回a除以b的商和余数                                                           | divmod(5,3)                        | (1,2)                   |
| pow(x,y[,z])            | 返回x**y。如果z有，则为pow(x,y)%z<br>>>> help(pow)<br>pow(base, exp, mod=None) | pow(2,10)<br>pow(2,10,10)          | 1024<br>4               |
| round(number[,ndigits]) | 四舍五入取整，如果ndigits则保留ndigits小数                                          | round(3.14159)<br>round(3.14159,4) | 3<br>3.1416             |
| max(arg1,arg2,*args)    | 取最大值                                                                  | max(1,7,3,15,14)                   | 15                      |
| min(arg1,arg2,*args)    | 取最小值                                                                  | min(1,7,3,15,14)                   | 1                       |
| sum(iterable[,start])   | 求和，start如果有，表示加上start                                                 | sum((1,2,3))<br>sum((1,2,3),44)    | 6<br>50                 |

## 例子：

- `max(iterable)`、`min(iterable)`、`sum(iterable)` 计算可迭代对象中所有元素最大值、最小值以及所有元素之和，
  - `sum` 只支持数值型元素的可迭代对象
  - `max` 和 `min` 则要求元素之间可比较大小。

```
>>> import random
>>> a = [random.randint(1,100) for i in range(10)]
>>> a
[72, 26, 80, 65, 34, 86, 19, 74, 52, 40]
>>> print(max(a), min(a), sum(a), sum(a)/len(a))
86 19 548 54.8
```